

Codemagic setup sheet — fill in the blanks

Copy this page, fill in each blank, and Phase 4 (iOS) + Phase 3 (Android) become paste-only. Generated for the operator · June 2026

This sheet matches the workflows in `codemagic.yaml`. Names in `code` must match *exactly* — they are the references the build file looks for. App ID is already set: `app.fabricflo.tracker`.

*How to read this: anything in **[BRACKETS]** is something you paste/create. "Where to get it" tells you the website + clicks. You never edit `codemagic.yaml` itself unless a name below says so.*

0. One-time connections in Codemagic

1. Go to <https://codemagic.io> → **Sign up with GitHub** → authorize the `fabric-flo` repository.
2. Open the app → it auto-detects `codemagic.yaml`. You'll run workflows `ios-release` and `android-release` from here.

1. Apple / iOS

1a. App Store Connect API key (lets Codemagic sign + upload for you)

Create at: **App Store Connect** → **Users and Access** → **Integrations (or Keys)** → **App Store Connect API** → +

What	Your value	Where to get it
Key file (<code>.p8</code>)	[DOWNLOAD AuthKey_XXXX.p8]	The "Download API Key" button (one time only — save it!)
Key ID	[_____]	Shown next to the key (e.g. 2X9ABC3DEF)
Issuer ID	[_____]	Top of the Keys page (a long UUID)

Access role	App Manager (or Admin)	Set when creating the key
-------------	-------------------------------	---------------------------

In Codemagic: Teams ▶ **Integrations** ▶ Apple Developer Portal / App Store Connect ▶ add the key. Give the integration this exact name so the build finds it:

Integration name: `FabricFloAppStoreKey`

This name matches `integrations: app_store_connect: FabricFloAppStoreKey` in `codemagic.yaml`. If you name it differently, change that one line to match.

1b. Create the app record in App Store Connect

App Store Connect → **Apps** → + → **New App**

Field	Value
Platform	iOS
Name	Fabric Flo
Bundle ID	app.fabricflo.tracker (register under Certificates ▶ Identifiers if not listed)
SKU	[anything unique, e.g. fabricflo-001]

After it's created, copy the **numeric Apple ID** (App Store Connect → your app → App Information → "Apple ID", a number like `6502345678`):

Variable	Your value	Used in
<code>APP_STORE_APPLE_ID</code>	<code>[_____]</code>	<code>codemagic.yaml</code> → <code>ios-release</code> → <code>vars</code> (replace the <code>0000000000</code> placeholder)

*Signing certificates/profiles: you do **nothing manual** — `distribution_type: app_store` + the API key let Codemagic create and manage them automatically.*

2. Android

2a. Upload keystore (signs the app; back this up forever)

If you don't have one yet, you can create it locally (Windows, in the project's `android` folder):

```
keytool -genkey -v -keystore fabricflo-upload.jks -keyalg RSA
-keysize 2048 -validity 10000 -alias upload
```

Save the file and the passwords in a password manager. **If you lose this, you can never update the app on Google Play.**

In Codemagic: App settings ▶ **Code signing identities** ▶ **Android keystores** ▶ upload it, with:

What	Your value	Notes
Keystore file	<code>[fabricflo-upload.jks]</code>	the file you generated
Keystore password	<code>[_____]</code>	from <code>keytool</code>
Key alias	<code>upload</code>	(or whatever you chose)
Key password	<code>[_____]</code>	from <code>keytool</code>
Reference name	<code>fabricflo_upload_keystore</code>	must match <code>codemagic.yaml</code>

2b. Google Play service account (lets Codemagic upload the `.aab`)

1. Google Play Console → **Setup** ▶ **API access** → link a Google Cloud project.
2. Create a **service account**, grant it **Release manager** permission in Play Console.
3. Download its **JSON key**.

In Codemagic: Teams ▶ Integrations ▶ Google Play ▶ upload the JSON. Then add it as a variable:

Variable	Your value	Notes
----------	------------	-------

GCPLOUD_SERVICE_ACCOUNT_CREDENTIALS	[paste JSON contents]	mark Secure ; used by <code>codemagic.yaml</code> Android publishing
-------------------------------------	-----------------------	---

First uploads sometimes must be done **by hand** in Play Console (Google requires the very first `.aab` manually). After that, Codemagic can publish to the **internal** track automatically.

3. Web/app environment (baked into both builds)

These are the same `VITE_*` values from your `.env` / Netlify. The build reads them at `npm run build`.

In Codemagic: App settings ▸ **Environment variables** ▸ create a group named exactly:

Group name: `fabricflo_web`

Variable	Your value	Secure?
VITE_SUPABASE_URL	[https://xxxx.supabase.co]	no
VITE_SUPABASE_ANON_KEY	[_____]	yes
VITE_FABRIC_FLO_BACKEND	normalized	no
VITE_PUBLIC_APP_URL	[https://your-site.netlify.app] (no trailing slash)	no
VITE_SUPPORT_EMAIL	[]	no
VITE_PRIVACY_EMAIL	[]	no

This group name matches `groups: - fabricflo_web` in `codemagic.yaml`. For Android, also create an (empty is fine) group named `fabricflo_android` to match the file.

4. Run it

Goal	In Codemagic
iOS to TestFlight	Run workflow <code>ios-release</code> → check email/TestFlight
Android to Play (internal)	Run workflow <code>android-release</code>
Submit iOS to App Store	After listing is complete, set <code>submit_to_app_store: true</code> in <code>codemagic.yaml</code> and re-run

Quick "did I set everything?" checklist

- ☐ GitHub connected; `codemagic.yaml` detected
- ☐ App Store Connect API key uploaded as `FabricFloAppStoreKey`
- ☐ App created in App Store Connect; `APP_STORE_APPLE_ID` filled into `codemagic.yaml`
- ☐ Android keystore uploaded as `fabricflo_upload_keystore` (and backed up!)
- ☐ Google Play service account JSON added (`G_CLOUD_SERVICE_ACCOUNT_CREDENTIALS`)
- ☐ `fabricflo_web` variable group filled (all `VITE_*`)
- ☐ `fabricflo_android` group exists (can be empty)
- ☐ First Android `.aab` uploaded manually in Play Console (Google's one-time requirement)

When these are checked, `ios-release` and `android-release` run end-to-end.

Fabric Flo — Film fabric & bag tracker for productions. Repo:

<https://github.com/KeatingMichael/fabric-flo>